GReinSS Tutorial — Speaker Notes

NCI Spring School on Algorithmic Cancer Biology · 30-minute slot

One block per slide. → NOTEBOOK slides are the live-demo hand-offs.

---

1. GReinSS

Hi everyone. Today: a hands-on tutorial on GReinSS — a method for a problem that shows up all over algorithmic cancer biology: inferring hidden combinatorial states from noisy, indirect measurements. We'll cover the idea, the one theorem that makes it work, and then train it live on your laptop. Goal: you leave able to apply it to your own problem.

2. The recurring problem in computational biology

The unifying pattern: a hidden discrete structure S, indirect observation X, and a KNOWN or partially-known likelihood Pr(X|S). Trees, CNV sets, isoforms — all fit. This is "self-supervised": the physics/biology of measurement is known; the state is not.

3. Two things we want

Panel a: hidden distribution over states S*, each emits an X. We model Pr(S|θ). Two problems: (1) LEARN θ from all observations jointly — the shared model couples them; (2) INFER the best state per observation. Everything today serves these two.

4. Why the usual tools struggle

EM: exact expectation needs summing over all states — only works for special structure (HMMs). VAE: great generative models, but the latent lives in a made-up $\mathbb{R}$^d, not your isoform space. Local search: per-observation, no sharing of statistical strength. Naive PG / GFlowNet are the closest cousins to what we do — and we'll see exactly why they fail.

5. The GReinSS idea

(no notes)

6. Generate states as trajectories

This is the RL move: represent a big discrete object as a path of small decisions. The policy is a neural net that at each step picks the next action. Any structure you can grow incrementally fits. We never enumerate S — we sample trajectories and average.

7. The one equation that matters

This is the whole method in one line. The numerator Pr(Xi|τ) is the usual "how well does this trajectory explain observation i". The DENOMINATOR Pr(Xi|θ) is the current model's total probability of that observation — it rescales each observation's contribution. Gradient is taken ONLY through log Pr(τ|θ);

the reward is treated as a constant each step, then recomputed after the update. That's the "dynamic" part.

8. Why the denominator? (intuition)

Key teaching moment. Without rescaling, τ1 has the highest raw reward, so naive PG puts ALL mass on it — but then X2 has zero probability and the joint likelihood is ZERO. With the denominator, as soon as τ1 gets probability its reward drops (it's dividing by its own success), so the policy is pushed to also cover X2. Equilibrium = the likelihood optimum. Note τ3 dies: it's dominated by τ2 for explaining X2. The method finds the RIGHT support.

9. The training loop

Emphasize the API surface: the user supplies (a) a generator and (b) Pr(X|S). That's it. The reward machinery, sampling, and gradient are provided. This is exactly what the notebook will show — you'll write those two functions and call train().

10. Off-policy learning (when on-policy is too slow)

Practical must-have for hard problems. Instead of blindly sampling from the policy, we tilt sampling toward states that actually fit each observation — provably the best proposal. In our biology applications this is where domain knowledge enters: a fast classical method proposes candidate states, and GReinSS refines the distribution over them. Keep this slide brief unless the audience asks.

11. → NOTEBOOK · Demo 1: Set reconstruction

SWITCH TO JUPYTER. Walk through: load observations → define Pr(X|S) (one line) → build generator net → train ~200 epochs live (watch the likelihood curve rise) → infer states → compare to naive thresholding. The punchline: GReinSS denoises using structure shared across observations, beating per-pixel rounding.

12. Results — simulations

Two combinatorial state types, same method. Left: graphs — GReinSS dominates especially when observations are information-poor (few walks). Right: sets — GReinSS is the only method that scales to large universes. GEM-based methods (VAE/autoregressive/diffusion) plateau; the closest RL cousins (naive PG, GFlowNet) fail. The reward rescaling is the difference.

13. → NOTEBOOK · Demo 2: Graph inference (pre-trained)

SWITCH TO JUPYTER (second section). Heavier model, so we ship a pre-trained checkpoint. Show: load model → simpleInference → compare predicted adjacency to the ground-truth graph we saved during pre-training → report F1 and visualize one graph. This mirrors the paper's Fig on graph inference but on your own generated instance with known truth.

14. Application — RNA isoforms beat RSEM

The payoff for this audience. Isoform quantification is a textbook latent-variable problem: short reads are indirect observations of full-length transcripts. RSEM is the standard EM tool GTEx ships.

Dropping GReinSS in — with a trivial Pr(X|S) — matches long-read ground truth far better. Panel c: on MBD2, GReinSS recovers the two true isoforms with near-correct proportions; RSEM splits mass across wrong isoforms. Panel d: distribution of (GReinSS - RSEM) error is shifted negative $\rightarrow$ GReinSS wins across the genome.

15. GReinSS already powers two cancer methods

GReinSS isn't just a new paper method — it's the generalization of machinery that already produced two cancer-genomics tools. If you work on trees, CNVs, isoforms, or any grow-able discrete structure with a known likelihood, this framework likely applies to you.

16. When should you reach for GReinSS?

Decision guide. The two hard requirements: an incremental generator and a likelihood. If you have those, the four-line recipe is all you need to start — exactly what the notebook demonstrates.

17. Thank you — let's build

Wrap up: the method is one reward formula with a clean theorem, it beats the standard tools on simulations and on real isoform data, and it's a drop-in for discrete latent-state problems in cancer genomics. Open the notebook and try it on your own Pr(X|S).